

LLM Workflows – Three Live Experiments

The reference architecture — Text-to-SQL, RAG QA, and Prompt Chaining — taken off the page and shipped as three deployed apps. Same three orchestration patterns, real vector search, real self-repair, real controls.

01 · SCHEMA-GROUNDED

Text-to-SQL Agent

Retrieval → generation → execution → self-repair

02 · VECTOR-GROUNDED

RAG-Based Q&A

3072-d embeddings, HNSW cosine, adjustable top-k

03 · DECOMPOSED

Prompt-Chain Summarizer

Six narrow steps, tone + length control

01

Text-to-SQL Agent

Ask your warehouse in plain English

► retrieval → generation → execution → repair

Embeds the question, retrieves the relevant tables from a schema catalog, writes a read-only SQLite query, runs it against a live in-browser warehouse — and repairs itself if the query fails.

WORKFLOW

1

Schema retrieval

The question is embedded and compared against 7 table descriptions; only the relevant tables are pulled into context.

2

Query generation

The LLM writes a single, read-only SQL query (SELECT / WITH only) grounded in the retrieved tables.

3

Execution

The query runs against a seeded demo warehouse — SQLite compiled to WASM, entirely in the browser.

4

Self-healing repair

Self-healing repair

A failed query becomes a second LLM call, error included — the agent regenerates and re-runs automatically.

SCHEMA CATALOG

customers

Signup date, country, marketing channel, loyalty tier

products

Category, unit price, cost, stock level

orders

Status, order date, shipping country

order_items

Product, units, unit price charged

payments

Method, amount, success flag

support_tickets

Topic, priority, satisfaction score, resolution time

web_sessions

Device, page views, duration, conversion

- VS REFERENCE** • Reference demo sends the full schema; this always retrieves a relevant subset by embedding similarity.
- Adds a repair loop — a failed query isn't a dead end, it's a second grounded call.

IMPLEMENTATION

text_to_sql.py

```
1 def generate_sql(question: str, tables: list) -> str:
2     system_prompt = f"""You are a SQL expert. Using only these tables,
3     write a single read-only SQLite query that answers the question.
4
5     Rules:
6     - SELECT or WITH only. No explanation, no markdown.
7     """
8     sql = call_llm(system_prompt, question)
9     return sql.replace("`sql", "").replace("`", "").strip()
10
11 def run_with_repair(conn, question, tables, retries=2):
12     sql = generate_sql(question, tables)
13     for attempt in range(retries):
14         try:
15             return execute(conn, sql)
16         except SqlError as err:
17             sql = repair_sql(question, sql, str(err))
18     return execute(conn, sql)
19
20 def repair_sql(question, bad_sql, error):
21     prompt = f"This query failed:\n{bad_sql}\nError: {error}\nFix it."
22     return call_llm(prompt, question).strip()
```

RAG-Based Question Answering

Retrieval-augmented question answering

►index → retrieve → generate

Indexes documents into a 3072-dimension vector space, retrieves the closest passages by cosine similarity, and generates answers grounded solely in what's retrieved.

WORKFLOW

1

Indexing

Documents are split into ~1,200-character overlapping chunks, embedded, and stored in an HNSW cosine index.

2

Retrieval

The question is embedded and matched against the index; top-k chunks return, k adjustable at 3 / 5 / 8.

3

Response generation

The LLM is instructed to answer using only the retrieved chunks — nothing from general knowledge.

- VS REFERENCE** • Reference demo uses TF-IDF, sparse keyword vectors. This uses dense 3072-d embeddings with HNSW cosine search — the semantic-matching production upgrade the reference calls for.
- Top-k is a live, user-adjustable control rather than a fixed default.

IMPLEMENTATION

rag_qa.py

```
1 def index_document(doc: str, chunk_size=1200, overlap=200):
2     chunks = split_overlapping(doc, chunk_size, overlap)
3     vectors = [embed(c) for c in chunks] # 3072-d embeddings
4     hnsw_index.add_batch(vectors, chunks)
5
6 def retrieve_chunks(question: str, top_k=5):
7     query_vec = embed(question)
8     hits = hnsw_index.search(query_vec, k=top_k, metric="cosine")
9     return [chunk_store[h.id] for h in hits]
10
11 def generate_answer(question: str, top_k=5):
12     chunks = retrieve_chunks(question, top_k)
13     context = "\n".join(f"- {c}" for c in chunks)
14     system_prompt = f"Answer ONLY using this context:\n{context}"
15     return call_llm(system_prompt, question)
```

live demo → neon-qna-glow.lovable.app

Prompt-Chain Summarizer

Multi-step summarization

►six narrow, inspectable links

One giant prompt hallucinates. This chain splits the job into six narrow prompts — each consuming the previous link's output, every intermediate artifact left open for inspection.

WORKFLOW

1

Normalize

Strip noise, fix structure, keep every fact.

2

Extract facts

Pull atomic claims, entities, and numbers.

3

Outline

Group the facts into a skeleton for the summary.

4

Draft

Write the first summary from the outline only.

5

Critique

Audit the draft for hallucination, omissions, and bloat.

6

Polish

Apply the critique and hit the target tone and length.

CONFIGURABLE OUTPUT

Tone

Neutral · Executive · Technical · Friendly

Length

Tight (~80 words) · Standard (~160 words) · Detailed (~320 words)

- VS REFERENCE** • Reference chains 4 steps (extract → group → summarize → combine). This extends to 6, adding Normalize up front and Critique before Polish — the fact list from step 2 is the ground truth Critique audits against.
- Tone and target length are live controls that steer Draft and Polish.

IMPLEMENTATION

summarization_chain.py

```
1 def summarize_via_chain(text: str, tone="neutral", length="standard"):
2     normalized = normalize(text)
3     facts = extract_facts(normalized)
```

```

4     outline = build_outline(facts)
5     draft = write_draft(outline, tone, length)
6     critique = critique_draft(draft, facts)
7     polished = polish(draft, critique, tone, length)
8     return {
9         "facts": facts,
10        "outline": outline,
11        "draft": draft,
12        "critique": critique,
13        "final_summary": polished,
14    }
15
16 def critique_draft(draft: str, facts: list) -> str:
17     prompt = f"Facts:\n{facts}\n\nDraft:\n{draft}\n"
18     prompt += "List any hallucinations, omissions, or bloat."
19     return call_llm(prompt, draft)

```

live demo → ask-any-data-33.lovable.app/summarize

Side-by-side

EXPERIMENT	DATA FED TO THE LLM	DISTINGUISHING FEATURE
Text-to-SQL Agent	Question + embedding-retrieved tables from a 7-table catalog	Self-healing: repairs a query that fails to execute
RAG-Based Q&A	Question + top-k chunks from a 3072-d HNSW cosine index	Adjustable top-k (3 / 5 / 8) at query time
Prompt-Chain Summarizer	Source text, then each of 6 sequential steps' own output	Adjustable tone + length; critique step audits for hallucination

Common foundation

All three experiments are the same underlying move — ground a call in the right data, or split a task into simpler sequential calls. Text-to-SQL grounds generation in a retrieved schema; RAG-based Q&A grounds it in retrieved document chunks; the summarizer decomposes one large task into six narrow, inspectable steps. What differs is not the model but what data reaches it and how many calls are chained — and each build pushes its pattern one step past the reference demo: real vector retrieval instead of TF-IDF, a repair loop instead of a single shot, a deeper six-link chain instead of four.